# Short Notes Guide - Integrating Appwrite Auth, Database, & React Context API

## Guide to Integrating Appwrite Authentication, Database, and React Context API

This guide details integrating Appwrite for authentication and database with React's Context API for global state. It covers a profile page and a "watch list" for movies, TV shows, seasons, and episodes.

---

## Table of Contents


1. **Setting up Appwrite Project**
    - 1.1. Creating a New Appwrite Project
    - 1.2. Configuring Platforms
    - 1.3. Creating Authentication
    - 1.4. Creating Database and Collections
2. **Integrating Appwrite SDK in React Native**
    - 2.1. Installation
    - 2.2. Appwrite Client Initialization
    - 2.3. Project Structure for Appwrite Services
3. **Implementing Appwrite Authentication**
    - 3.1. User Registration (Sign Up)
    - 3.2. User Login (Sign In)
    - 3.3. Getting Current User Session
    - 3.4. User Logout
    - 3.5. Handling Authentication State (Loading, Errors)
4. **Implementing Appwrite Database for Watchlist**
    - 4.1. Creating Watchlist Collection
    - 4.2. Adding a Movie to Watchlist
    - 4.3. Removing a Movie from Watchlist
    - 4.4. Fetching User's Watchlist
    - 4.5. Checking if a Movie is in Watchlist

5. **React Context API for Global State Management**
    - 5.1. Understanding Context API
    - 5.2. Creating an Auth Context
    - 5.3. Creating a Watchlist Context
    - 5.4. Combining Context Providers
6. **Integrating into UI Components and Screens**
    - 6.1. Updating Navigation (Conditional Rendering based on Auth)
    - 6.2. Creating a Profile Screen
    - 6.3. Integrating Watchlist Functionality into Details Screens and Episode Items
    - 6.4. Displaying Watchlist on WatchListScreen
7. **Final Thoughts and Best Practices**

---

# 1. Setting up Appwrite Project

Set up your Appwrite backend before coding.

## 1.1. Creating a New Appwrite Project

1. **Install Appwrite:** Install Appwrite (see https://appwrite.io/docs/installation).
2. **Access Console:** Open your Appwrite console (e.g., `http://localhost`).
3. **Create Project:** Create a project (e.g., "MovieListingApp"). Note its **Project ID**.

## 1.2. Configuring Platforms

Inform Appwrite about connecting platforms.

1. **Add Platform:** Go to "Overview" → "Add Platform".
2. **Select "Flutter App" or "React Native App":** Choose "Flutter App" or a generic "Client" for React Native.
3. **Enter Application ID:** Use your React Native app's package name (e.g., `com.yourcompany.movielisting`), found in `app.json` or `package.json`.

## 1.3. Creating Authentication

Appwrite handles user authentication.

1. **Users:** Find registered users under "Auth" → "Users" in the console.
2. **Settings:** Configure authentication options (e.g., email verification, password policies, OAuth) in "Auth" → "Settings". Email/password is the current focus.

## 1.4. Creating Database and Collections

Appwrite Database stores user watchlists.

1. **Create Database:** Go to "Database", "Create Database" (e.g., "MovieData"). Note **Database ID**.
2. **Create Collection (Watchlist):**
   - In the database, "Create Collection", name it "Watchlist". Note **Collection ID**.
   - **Permissions:** Crucial.
     - **Read/Write Access:** Grant "Document Permissions" to "users".
     - *Important:* For "Watchlist" collection, set "Permissions" to allow "create", "read", "update", "delete" for `users`. Document-level permissions in code will restrict users to *their own* items.
3. **Add Attributes to Watchlist Collection:**
   - In "Watchlist" collection, go to "Attributes".
   - Add:
     - `userId`: `String`, `true` (Links to user).
     - `itemId`: `String`, `true` (Unique ID: movie, TV show, season, episode).
     - `itemTitle`: `String`, `true` (Title: movie, TV show, season, episode).
     - `itemPosterPath`: `String`, `false` (Poster/thumbnail URL).
     - `itemReleaseDate`: `String`, `false` (Release/air date).
     - `mediaType`: `String`, `true` (e.g., 'movie', 'tv', 'season', 'episode').
     - `parentItemId`: `String`, `false` (Parent ID for season/episode).
     - `seasonNumber`: `Integer`, `false` (Season number).
     - `episodeNumber`: `Integer`, `false` (Episode number).

---

## 2. Integrating Appwrite SDK in React Native

Prepare your React Native app for Appwrite communication.

### 2.1. Installation

Install the Appwrite SDK in your project root:

```
npm install appwrite
# or
yarn add appwrite
```

### 2.2. Appwrite Client Initialization

Create an Appwrite client configuration file.

1. **Create `src/api/appwrite.js`:**

```
// src/api/appwrite.js
import { Client, Account, Databases } from 'appwrite';

export const appwriteConfig = {
  endpoint: process.env.APPWRITE_ENDPOINT || 'YOUR_APPWRITE_ENDPOINT', // e.g., '<http://localhost/v1>'
  projectId: process.env.APPWRITE_PROJECT_ID || 'YOUR_APPWRITE_PROJECT_ID',
  databaseId: process.env.APPWRITE_DATABASE_ID || 'YOUR_APPWRITE_DATABASE_ID',
  watchlistCollectionId: process.env.APPWRITE_WATCHLIST_COLLECTION_ID || 'YOUR_APPWRITE_WATCHLIST_COLLECTION_ID',
};

const client = new Client()
  .setEndpoint(appwriteConfig.endpoint)
  .setProject(appwriteConfig.projectId);

const account = new Account(client);
const databases = new Databases(client);

export { client, account, databases };
```

**Note:** Replace placeholders with your Appwrite console values. Use environment variables (`process.env.APPWRITE_...`) for security.

### 2.3. Project Structure for Appwrite Services

Organize your `src/` directory for new Appwrite services and context.

```
src/
├── api/
│   ├── apiClient.js
│   ├── apiConfig.js
│   └── appwrite.js       ← New Appwrite client
├── auth/               ← New Appwrite auth logic
│   └── authService.js
├── components/
├── context/            ← New React Context API providers
│   ├── AuthContext.js
│   └── WatchlistContext.js
├── database/           ← New Appwrite database logic
│   └── watchlistService.js
├── navigation/
└── screens/
    ├── HomeScreen.js
```

```
    ├── MovieDetailsScreen.js
    ├── MoviesScreen.js
    ├── ProfileScreen.js   ← New User profile screen
    ├── SearchScreen.js
    ├── TvShowDetailsScreen.js
    ├── TVShowsScreen.js
    ├── WatchListScreen.js
    ├── WatchTrailerScreen.js
    └── WatchVideoScreen.js
```

---

## 3. Implementing Appwrite Authentication

Create a service file for Appwrite authentication logic.

### 3.1. User Registration (Sign Up)

In `src/auth/authService.js`:

```
// src/auth/authService.js
import { ID } from 'appwrite';
import { account } from '../api/appwrite'; // Import the initialized account object

const AuthService = {
  async register(email, password, name) {
    try {
      const user = await account.create(ID.unique(), email, password, name);
      console.log('User registered:', user);
      return user;
    } catch (error) {
      console.error('Registration error:', error);
      throw error;
    }
  },

  // ... other auth methods
};

export default AuthService;
```

**Example Usage (`SignUpScreen.js`):**

```
// screens/SignUpScreen.js (example)
import React, { useState } from 'react';
import { View, Text, TextInput, Button, Alert } from 'react-native';
import AuthService from '../src/auth/authService';
```

```
const SignUpScreen = ({ navigation }) => {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [name, setName] = useState('');
  const [loading, setLoading] = useState(false);

  const handleSignUp = async () => {
    if (!email || !password || !name) {
      Alert.alert('Error', 'Please fill all fields.');
      return;
    }
    setLoading(true);
    try {
      await AuthService.register(email, password, name);
      Alert.alert('Success', 'Account created successfully! Please log in.');
      navigation.navigate('SignIn'); // Navigate to login screen
    } catch (error) {
      Alert.alert('Error', error.message || 'Something went wrong during registration.');
    } finally {
      setLoading(false);
    }
  };

  return (
    <View style={{ flex: 1, justifyContent: 'center', padding: 20 }}>
      <Text style={{ fontSize: 24, marginBottom: 20 }}>Sign Up</Text>
      <TextInput
        placeholder="Name"
        value={name}
        onChangeText={setName}
        style={{ borderWidth: 1, borderColor: 'gray', padding: 10, marginBottom: 10 }}
      />
      <TextInput
        placeholder="Email"
        value={email}
        onChangeText={setEmail}
        keyboardType="email-address"
        autoCapitalize="none"
        style={{ borderWidth: 1, borderColor: 'gray', padding: 10, marginBottom: 10 }}
      />
      <TextInput
        placeholder="Password"
        value={password}
        onChangeText={setPassword}
```

```
      secureTextEntry
      style={{ borderWidth: 1, borderColor: 'gray', padding: 10, marginBottom: 20 }}
    />
    <Button title={loading ? 'Signing Up...' : 'Sign Up'} onPress={handleSignUp} disabled={loading}
/>
    <Button title="Already have an account? Sign In" onPress={() => navigation.navigate('SignIn')} /
>
  </View>
 );
};

export default SignUpScreen;
```

### 3.2. User Login (Sign In)

Add to `src/auth/authService.js`:

```
// src/auth/authService.js (continued)
// ...
const AuthService = {
 // ... register method

 async login(email, password) {
  try {
   const session = await account.createEmailPasswordSession(email, password);
   console.log('User logged in:', session);
   return session;
  } catch (error) {
   console.error('Login error:', error);
   throw error;
  }
 },

 // ... other auth methods
};

export default AuthService;
```

**Example Usage (`SignInScreen.js`):**

```
// screens/SignInScreen.js (example)
import React, { useState } from 'react';
import { View, Text, TextInput, Button, Alert } from 'react-native';
import AuthService from '../src/auth/authService';
import { useAuth } from '../src/context/AuthContext'; // We'll create this later
```

```
const SignInScreen = ({ navigation }) => {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [loading, setLoading] = useState(false);
  const { login: authContextLogin } = useAuth(); // Use context for global state update

  const handleSignIn = async () => {
    if (!email || !password) {
      Alert.alert('Error', 'Please enter email and password.');
      return;
    }
    setLoading(true);
    try {
      const session = await AuthService.login(email, password);
      await authContextLogin(session); // Update global auth state
      Alert.alert('Success', 'Logged in successfully!');
      // Navigate to the main app or a protected screen
      navigation.replace('Home'); // Example: Replace current screen with Home
    } catch (error) {
      Alert.alert('Error', error.message || 'Login failed. Please check your credentials.');
    } finally {
      setLoading(false);
    }
  };

  return (
    <View style={{ flex: 1, justifyContent: 'center', padding: 20 }}>
      <Text style={{ fontSize: 24, marginBottom: 20 }}>Sign In</Text>
      <TextInput
        placeholder="Email"
        value={email}
        onChangeText={setEmail}
        keyboardType="email-address"
        autoCapitalize="none"
        style={{ borderWidth: 1, borderColor: 'gray', padding: 10, marginBottom: 10 }}
      />
      <TextInput
        placeholder="Password"
        value={password}
        onChangeText={setPassword}
        secureTextEntry
        style={{ borderWidth: 1, borderColor: 'gray', padding: 10, marginBottom: 20 }}
      />
      <Button title={loading ? 'Signing In...' : 'Sign In'} onPress={handleSignIn} disabled={loading} />
```

```
    <Button title="Don't have an account? Sign Up" onPress={() => navigation.navigate('SignUp')} /
>
   </View>
  );
};

export default SignInScreen;
```

### 3.3. Getting Current User Session

Add to `src/auth/authService.js`:

```
// src/auth/authService.js (continued)
// ...
const AuthService = {
  // ... register, login methods

  async getCurrentUser() {
    try {
      const user = await account.get(); // Retrieves the current logged-in user
      console.log('Current user:', user);
      return user;
    } catch (error) {
      console.log('No current user or session expired:', error.message);
      return null;
    }
  },

  // ... other auth methods
};

export default AuthService;
```

### 3.4. User Logout

Add to `src/auth/authService.js`:

```
// src/auth/authService.js (continued)
// ...
const AuthService = {
  // ... register, login, getCurrentUser methods

  async logout() {
    try {
      await account.deleteSession('current'); // Deletes the current session
```

```
      console.log('User logged out');
      return true;
    } catch (error) {
      console.error('Logout error:', error);
      throw error;
    }
  },
};

export default AuthService;
```

### 3.5. Handling Authentication State (Loading, Errors)

Managed by `AuthContext` (created later); service functions handle API calls and errors.

---

## 4. Implementing Appwrite Database for Watchlist

Handle logic for "Watchlist" collection interaction.

### 4.1. Creating Watchlist Collection (Done in Appwrite Console)

(As in 1.4) Ensure "Watchlist" collection has `userId`, `itemId`, `itemTitle`, `itemPosterPath`, `itemReleaseDate`, `mediaType`, `parentItemId`, `seasonNumber`, and `episodeNumber`.

### 4.2. Adding an Item to Watchlist

Create `src/database/watchlistService.js`:

```
// src/database/watchlistService.js
import { ID, Permission, Role } from 'appwrite';
import { databases, appwriteConfig } from '../api/appwrite';

const WatchlistService = {
  async addItemToWatchlist(userId, itemData) {
    try {
      const { itemId, itemTitle, itemPosterPath, itemReleaseDate, mediaType, parentItemId, seasonNu
mber, episodeNumber } = itemData;
      const document = await databases.createDocument(
        appwriteConfig.databaseId,
        appwriteConfig.watchlistCollectionId,
        ID.unique(),
        {
          userId,
          itemId,
          itemTitle,
          itemPosterPath,
```

```
        itemReleaseDate,
        mediaType,
        parentItemId,
        seasonNumber,
        episodeNumber,
      },
      [
        Permission.read(Role.user(userId)),
        Permission.write(Role.user(userId)),
        Permission.update(Role.user(userId)),
        Permission.delete(Role.user(userId)),
      ]
    );
    console.log('Item added to watchlist:', document);
    return document;
  } catch (error) {
    console.error('Error adding item to watchlist:', error);
    throw error;
  }
},

// ... other watchlist methods
};

export default WatchlistService;
```

**Permissions Explanation:**
Document-level permissions (read, write, delete for `Role.user(userId)`) ensure users manage only their watchlist items, overriding broader collection permissions.

## 4.3. Removing an Item from Watchlist

Add to `src/database/watchlistService.js`:

```
// src/database/watchlistService.js (continued)
import { Query } from 'appwrite';
// ...
const WatchlistService = {
  // ... addItemToWatchlist method

  async removeItemFromWatchlist(userId, itemId, mediaType) {
    try {
      // First, find the document ID for the item in the user's watchlist
      const response = await databases.listDocuments(
        appwriteConfig.databaseId,
```

```
      appwriteConfig.watchlistCollectionId,
      [
        Query.equal('userId', userId),
        Query.equal('itemId', itemId),
        Query.equal('mediaType', mediaType), // Important for uniqueness
      ]
    );

    if (response.documents.length > 0) {
      const documentId = response.documents[0].$id;
      await databases.deleteDocument(
        appwriteConfig.databaseId,
        appwriteConfig.watchlistCollectionId,
        documentId
      );
      console.log('Item removed from watchlist:', documentId);
      return true;
    } else {
      console.warn('Item not found in watchlist for this user.');
      return false;
    }
  } catch (error) {
    console.error('Error removing item from watchlist:', error);
    throw error;
  }
},

// ... other watchlist methods
};

export default WatchlistService;
```

### 4.4. Fetching User's Watchlist

Add to `src/database/watchlistService.js`:

```
// src/database/watchlistService.js (continued)
import { Query } from 'appwrite';
// ...
const WatchlistService = {
  // ... addMovie, removeMovie methods

  async getWatchlist(userId) {
    try {
      const response = await databases.listDocuments(
```

```
      appwriteConfig.databaseId,
      appwriteConfig.watchlistCollectionId,
      [
        Query.equal('userId', userId),
        Query.orderDesc('$createdAt'), // Optional: order by most recently added
      ]
    );
    console.log('User watchlist:', response.documents);
    return response.documents;
  } catch (error) {
    console.error('Error fetching watchlist:', error);
    throw error;
  }
},

// ... other watchlist methods
};

export default WatchlistService;
```

### 4.5. Checking if an Item is in Watchlist

Add to `src/database/watchlistService.js`:

```
// src/database/watchlistService.js (continued)
import { Query } from 'appwrite';
// ...
const WatchlistService = {
  // ... addItemToWatchlist, removeItemFromWatchlist, getWatchlist methods

  async isItemInWatchlist(userId, itemId, mediaType) {
    try {
      const response = await databases.listDocuments(
        appwriteConfig.databaseId,
        appwriteConfig.watchlistCollectionId,
        [
          Query.equal('userId', userId),
          Query.equal('itemId', itemId),
          Query.equal('mediaType', mediaType), // Important for uniqueness
          Query.limit(1), // We only need to know if at least one exists
        ]
      );
      return response.documents.length > 0;
    } catch (error) {
      console.error('Error checking item in watchlist:', error);
```

```
    throw error;
  }
 },
};

export default WatchlistService;
```

---

## 5. React Context API for Global State Management

Context API manages global states (auth, watchlist) without prop-drilling.

### 5.1. Understanding Context API

- `createContext`: Creates a Context object.
- `Provider`: Component supplying context `value` to descendants.
- `useContext`: Hook to read current context value.

### 5.2. Creating an Auth Context

This context manages the current user object and authentication state (login, logout).

Create `src/context/AuthContext.js`:

```
// src/context/AuthContext.js
import React, { createContext, useContext, useState, useEffect } from 'react';
import AuthService from '../auth/authService';

const AuthContext = createContext(null);

export const AuthProvider = ({ children }) => {
 const [user, setUser] = useState(null);
 const [loading, setLoading] = useState(true);

 useEffect(() => {
  const checkUser = async () => {
   try {
    const currentUser = await AuthService.getCurrentUser();
    setUser(currentUser);
   } catch (error) {
    console.error('Failed to fetch current user on mount:', error);
    setUser(null);
   } finally {
    setLoading(false);
   }
  };
```

```
    checkUser();
  }, []);

  const login = async (session) => {
    // After successful login from AuthService, fetch user details
    const currentUser = await AuthService.getCurrentUser();
    setUser(currentUser);
  };

  const logout = async () => {
    try {
      await AuthService.logout();
      setUser(null);
    } catch (error) {
      console.error('Logout failed:', error);
      throw error;
    }
  };

  return (
    <AuthContext.Provider value={{ user, loading, login, logout, isAuthenticated: !!user }}>
      {children}
    </AuthContext.Provider>
  );
};

export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within an AuthProvider');
  }
  return context;
};
```

### 5.3. Creating a Watchlist Context

This context manages the user's watchlist (add, remove, check items).

Create `src/context/WatchlistContext.js`:

```
// src/context/WatchlistContext.js
import React, { createContext, useContext, useState, useEffect, useCallback } from 'react';
import WatchlistService from '../database/watchlistService';
import { useAuth } from './AuthContext'; // To get the current user ID
```

```
const WatchlistContext = createContext(null);

export const WatchlistProvider = ({ children }) => {
  const { user, isAuthenticated, loading: authLoading } = useAuth();
  const [watchlist, setWatchlist] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  const fetchWatchlist = useCallback(async () => {
    if (!isAuthenticated || !user?.$id) {
      setWatchlist([]);
      setLoading(false);
      return;
    }

    setLoading(true);
    setError(null);
    try {
      const items = await WatchlistService.getWatchlist(user.$id);
      setWatchlist(items);
    } catch (err) {
      console.error('Failed to fetch watchlist:', err);
      setError(err);
    } finally {
      setLoading(false);
    }
  }, [isAuthenticated, user?.$id]);

  useEffect(() => {
    if (!authLoading) { // Only fetch watchlist once auth state is resolved
      fetchWatchlist();
    }
  }, [authLoading, fetchWatchlist]);

  const addItemToWatchlist = async (itemData) => {
    if (!isAuthenticated || !user?.$id) {
      throw new Error('User not authenticated.');
    }
    setLoading(true); // Indicate action is in progress
    try {
      const newItem = await WatchlistService.addItemToWatchlist(user.$id, itemData);
      setWatchlist((prev) => [newItem, ...prev]); // Add new item to the top
      return newItem;
    } catch (err) {
      setError(err);
```

```
    throw err;
  } finally {
    setLoading(false);
  }
};

const removeItemFromWatchlist = async (itemId, mediaType) => {
  if (!isAuthenticated || !user?.$id) {
    throw new Error('User not authenticated.');
  }
  setLoading(true); // Indicate action is in progress
  try {
    const success = await WatchlistService.removeItemFromWatchlist(user.$id, itemId, mediaTyp
e);
    if (success) {
      setWatchlist((prev) => prev.filter((item) => !(item.itemId === itemId && item.mediaType === me
diaType)));
    }
    return success;
  } catch (err) {
    setError(err);
    throw err;
  } finally {
    setLoading(false);
  }
};

const isItemInWatchlist = useCallback((itemId, mediaType) => {
  return watchlist.some((item) => item.itemId === itemId && item.mediaType === mediaType);
}, [watchlist]);

return (
  <WatchlistContext.Provider
    value={{
      watchlist,
      loading,
      error,
      addItemToWatchlist,
      removeItemFromWatchlist,
      isItemInWatchlist,
      refreshWatchlist: fetchWatchlist, // Allow components to manually refresh
    }}
  >
    {children}
  </WatchlistContext.Provider>
```

```
  );
};

export const useWatchlist = () => {
  const context = useContext(WatchlistContext);
  if (!context) {
    throw new Error('useWatchlist must be used within a WatchlistProvider');
  }
  return context;
};
```

### 5.4. Combining Context Providers

Wrap your app with `AuthProvider` and `WatchlistProvider`. `WatchlistProvider` nests inside `AuthProvider` due to its dependency.

Modify your `App.js`:

```
// App.js
import React from 'react';
import AppNavigator from './src/navigation/AppNavigator'; // Your main navigator
import { AuthProvider } from './src/context/AuthContext';
import { WatchlistProvider } from './src/context/WatchlistContext';
import { StatusBar } from 'react-native';

export default function App() {
  return (
    <AuthProvider>
      <WatchlistProvider>
        <StatusBar barStyle="light-content" />
        <AppNavigator />
      </WatchlistProvider>
    </AuthProvider>
  );
}
```

---

## 6. Integrating into UI Components and Screens

Integrate functionality into your application's UI.

### 6.1. Updating Navigation (Conditional Rendering based on Auth)

The app should function for non-logged-in users. However, sensitive screens like 'Profile' and 'Watchlist' will require authentication.
Modify `src/navigation/AppNavigator.js` (or root navigator) to conditionally render different navigators based on the user's authentication state:

```
// src/navigation/AppNavigator.js (example)
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import BottomTabNavigator from './BottomTabNavigator'; // Your existing main tabs
import { useAuth } from '../context/AuthContext';
import SignInScreen from '../screens/SignInScreen'; // Create this screen
import SignUpScreen from '../screens/SignUpScreen'; // Create this screen
import LoadingScreen from '../screens/LoadingScreen'; // A simple loading screen

const Stack = createNativeStackNavigator();

const AppNavigator = () => {
  const { isAuthenticated, loading } = useAuth();

  if (loading) {
    return <LoadingScreen />; // Show a loading indicator while checking auth status
  }

  return (
    <NavigationContainer>
      <Stack.Navigator screenOptions={{ headerShown: false }}>
        {isAuthenticated ? (
          // User is authenticated, show main app
          <Stack.Screen name="Main" component={BottomTabNavigator} />
        ) : (
          // User is not authenticated, show auth flow
          <>
            <Stack.Screen name="SignIn" component={SignInScreen} />
            <Stack.Screen name="SignUp" component={SignUpScreen} />
          </>
        )}
      </Stack.Navigator>
    </NavigationContainer>
  );
};

export default AppNavigator;
```

`screens/LoadingScreen.js`:

```
// screens/LoadingScreen.js
import React from 'react';
import { View, ActivityIndicator, Text, StyleSheet } from 'react-native';
```

```
const LoadingScreen = () => {
  return (
    <View style={styles.container}>
      <ActivityIndicator size="large" color="#0000ff" />
      <Text style={styles.text}>Loading...</Text>
    </View>
  );
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#fff',
  },
  text: {
    marginTop: 10,
    fontSize: 18,
    color: '#333',
  },
});

export default LoadingScreen;
```

Create `SignInScreen.js` and `SignUpScreen.js` (see section 3).

### 6.2. Creating a Profile Screen

This screen displays user information and a logout button if the user is authenticated. If not authenticated, it should prompt the user to sign in or register.

Modify `src/screens/ProfileScreen.js` to conditionally render its content:

```
// src/screens/ProfileScreen.js
import React from 'react';
import { View, Text, Button, StyleSheet, Alert } from 'react-native';
import { useAuth } from '../context/AuthContext';
import { useNavigation } from '@react-navigation/native';

const ProfileScreen = () => {
  const { user, logout } = useAuth();
  const navigation = useNavigation();

  const handleLogout = async () => {
```

```
    try {
      await logout();
      Alert.alert('Success', 'You have been logged out.');
      // Navigation will automatically update due to AuthContext change
    } catch (error) {
      Alert.alert('Error', error.message || 'Logout failed.');
    }
  };

  if (!user) {
    return (
      <View style={styles.container}>
        <Text style={styles.title}>Not Logged In</Text>
        <Button title="Sign In" onPress={() => navigation.navigate('SignIn')} />
      </View>
    );
  }

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Profile</Text>
      <Text style={styles.infoText}>Name: {user.name || 'N/A'}</Text>
      <Text style={styles.infoText}>Email: {user.email || 'N/A'}</Text>
      <Text style={styles.infoText}>User ID: {user.$id || 'N/A'}</Text>
      <View style={styles.buttonContainer}>
        <Button title="Logout" onPress={handleLogout} color="red" />
      </View>
    </View>
  );
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 20,
    backgroundColor: '#f0f0f0',
  },
  title: {
    fontSize: 28,
    fontWeight: 'bold',
    marginBottom: 30,
    color: '#333',
  },
```

```
  infoText: {
    fontSize: 18,
    marginBottom: 10,
    color: '#555',
    textAlign: 'center',
  },
  buttonContainer: {
    marginTop: 30,
    width: '80%',
  },
});

export default ProfileScreen;
```

Add `ProfileScreen` to `BottomTabNavigator.js` or `StackNavigators.js`.

**Example `BottomTabNavigator.js` modification:**

```
// src/navigation/BottomTabNavigator.js (example)
import React from 'react';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
// Import your existing screens
import HomeScreen from '../screens/HomeScreen';
import MoviesScreen from '../screens/MoviesScreen';
import TVShowsScreen from '../screens/TVShowsScreen';
import SearchScreen from '../screens/SearchScreen';
import WatchListScreen from '../screens/WatchListScreen'; // New screen for watchlist
import ProfileScreen from '../screens/ProfileScreen'; // New Profile Screen
import { Ionicons } from '@expo/vector-icons'; // Assuming you use Expo/Ionicons

const Tab = createBottomTabNavigator();

const BottomTabNavigator = () => {
  return (
    <Tab.Navigator
      screenOptions={({ route }) => ({
        tabBarIcon: ({ focused, color, size }) => {
          let iconName;
          if (route.name === 'Home') {
            iconName = focused ? 'home' : 'home-outline';
          } else if (route.name === 'Movies') {
            iconName = focused ? 'film' : 'film-outline';
          } else if (route.name === 'TV Shows') {
            iconName = focused ? 'tv' : 'tv-outline';
          } else if (route.name === 'Search') {
```

```
          iconName = focused ? 'search' : 'search-outline';
        } else if (route.name === 'Watchlist') {
          iconName = focused ? 'bookmark' : 'bookmark-outline';
        } else if (route.name === 'Profile') { // Add profile tab
          iconName = focused ? 'person' : 'person-outline';
        }
        return <Ionicons name={iconName} size={size} color={color} />;
      },
      tabBarActiveTintColor: 'tomato',
      tabBarInactiveTintColor: 'gray',
      headerShown: false, // Hide header for tab screens
    })}
  >
    <Tab.Screen name="Home" component={HomeScreen} />
    <Tab.Screen name="Movies" component={MoviesScreen} />
    <Tab.Screen name="TV Shows" component={TVShowsScreen} />
    <Tab.Screen name="Search" component={SearchScreen} />
    <Tab.Screen name="Watchlist" component={WatchListScreen} />
    <Tab.Screen name="Profile" component={ProfileScreen} /> {/* Add Profile Screen */}
  </Tab.Navigator>
 );
};

export default BottomTabNavigator;
```

### 6.3. Integrating Watchlist Functionality into Details Screens and Episode Items

The "Add to Watchlist" button should be present on `MovieDetailsScreen.js`, `TvShowDetailsScreen.js`, and for individual episode items displayed within `TvShowDetailsScreen` (e.g., in a FlatList). When a non-logged-in user attempts to add an item, an alert stating "You need to be logged in to add items to your list" should be displayed. The `ItemDetailsScreen` component can be adapted to serve as a generic details screen for movies and TV shows, handling the display of the watchlist button.

```
// src/screens/MovieDetailsScreen.js (example modification)
import React, { useState, useEffect } from 'react';
import { View, Text, ScrollView, Image, StyleSheet, TouchableOpacity, ActivityIndicator, Alert } fro
m 'react-native';
import { useRoute } from '@react-navigation/native';
// Assume you have an API client to fetch movie/tv details (e.g., TMDB)
import { fetchMovieDetails, fetchTvShowDetails } from '../api/apiClient'; // Assuming you have fetch
TvShowDetails

import { useAuth } from '../context/AuthContext';
import { useWatchlist } from '../context/WatchlistContext';
import { Ionicons } from '@expo/vector-icons';
```

```
const ItemDetailsScreen = () => { // Renamed for clarity, can remain MovieDetailsScreen
  const route = useRoute();
  const { itemId, mediaType, parentItemId, seasonNumber, episodeNumber } = route.params; // Get item ID and type from navigation params

  const { isAuthenticated } = useAuth();
  const {
    watchlist,
    addItemToWatchlist,
    removeItemFromWatchlist,
    isItemInWatchlist,
    loading: watchlistLoading,
  } = useWatchlist();

  const [itemDetails, setItemDetails] = useState(null);
  const [loading, setLoading] = useState(true);
  const [isAddingRemoving, setIsAddingRemoving] = useState(false);

  const isInWatchlist = isItemInWatchlist(String(itemId), mediaType); // Convert itemId to string for consistency with Appwrite

  useEffect(() => {
    const getDetails = async () => {
      setLoading(true);
      try {
        let details;
        if (mediaType === 'movie') {
          details = await fetchMovieDetails(itemId);
        } else if (mediaType === 'tv') {
          details = await fetchTvShowDetails(itemId);
        }
        // Add logic for 'season' and 'episode' if you want details for them
        setItemDetails(details);
      } catch (error) {
        console.error('Error fetching item details:', error);
        Alert.alert('Error', 'Failed to fetch item details.');
      } finally {
        setLoading(false);
      }
    };
    getDetails();
  }, [itemId, mediaType]);

  const handleWatchlistToggle = async () => {
```

```
    if (!isAuthenticated) {
      Alert.alert('Login Required', 'Please log in to manage your watchlist.');
      return;
    }

    setIsAddingRemoving(true);
    try {
      if (isInWatchlist) {
        await removeItemFromWatchlist(String(itemId), mediaType);
        Alert.alert('Removed', `${itemDetails?.title || itemDetails?.name} removed from watchlist.`);
      } else {
        const itemData = {
          itemId: String(itemId),
          itemTitle: itemDetails?.title || itemDetails?.name || `Season ${seasonNumber} Episode ${epis
odeNumber}`, // Adjust based on mediaType
          itemPosterPath: itemDetails?.poster_path || itemDetails?.still_path,
          itemReleaseDate: itemDetails?.release_date || itemDetails?.first_air_date || itemDetails?.air_da
te,
          mediaType: mediaType,
          parentItemId: parentItemId, // Will be undefined for movies/tv, set for seasons/episodes
          seasonNumber: seasonNumber,
          episodeNumber: episodeNumber,
        };
        await addItemToWatchlist(itemData);
        Alert.alert('Added', `${itemDetails?.title || itemDetails?.name || `Season ${seasonNumber} Epis
ode ${episodeNumber}`} added to watchlist!`);
      }
    } catch (error) {
      Alert.alert('Error', error.message || 'Could not update watchlist.');
      console.error('Watchlist update error:', error);
    } finally {
      setIsAddingRemoving(false);
    }
  };

  if (loading || (!itemDetails && !error)) {
    return (
      <View style={styles.loadingContainer}>
        <ActivityIndicator size="large" color="#0000ff" />
      </View>
    );
  }

  return (
    <ScrollView style={styles.container}>
```

```
    {(itemDetails.poster_path || itemDetails.still_path) && (
      <Image
        source={{ uri: `https://image.tmdb.org/t/p/w500${itemDetails.poster_path || itemDetails.still_
path}` }}
        style={styles.poster}
      />
    )}
    <View style={styles.detailsContainer}>
      <Text style={styles.title}>{itemDetails.title || itemDetails.name}</Text>
      <Text style={styles.overview}>{itemDetails.overview || itemDetails.description}</Text>
      <Text style={styles.info}>
        Release Date: {itemDetails.release_date || itemDetails.first_air_date || itemDetails.air_date}
      </Text>
      {itemDetails.vote_average && <Text style={styles.info}>Rating: {itemDetails.vote_average?.toF
ixed(1)} / 10</Text>}
      {mediaType === 'season' && seasonNumber && <Text style={styles.info}>Season: {seasonN
umber}</Text>}
      {mediaType === 'episode' && episodeNumber && <Text style={styles.info}>Episode: {episode
Number}</Text>}


      {/* Watchlist Button */}
      <TouchableOpacity
        style={[styles.watchlistButton, isInWatchlist ? styles.watchlistButtonActive : {}]}
        onPress={handleWatchlistToggle}
        disabled={isAddingRemoving || watchlistLoading}
      >
        {isAddingRemoving || watchlistLoading ? (
          <ActivityIndicator color="#fff" />
        ) : (
          <>
            <Ionicons
              name={isInWatchlist ? 'bookmark' : 'bookmark-outline'}
              size={24}
              color="#fff"
            />
            <Text style={styles.watchlistButtonText}>
              {isInWatchlist ? 'Remove from Watchlist' : 'Add to Watchlist'}
            </Text>
          </>
        )}
      </TouchableOpacity>

      {/* ... other details like genres, cast, etc. */}
    </View>
```

```
    </ScrollView>
  );
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#000',
  },
  loadingContainer: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#000',
  },
  poster: {
    width: '100%',
    height: 500,
    resizeMode: 'cover',
  },
  detailsContainer: {
    padding: 15,
  },
  title: {
    fontSize: 28,
    fontWeight: 'bold',
    color: '#fff',
    marginBottom: 10,
  },
  overview: {
    fontSize: 16,
    color: '#ccc',
    marginBottom: 15,
    lineHeight: 24,
  },
  info: {
    fontSize: 14,
    color: '#aaa',
    marginBottom: 5,
  },
  watchlistButton: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'center',
    backgroundColor: '#4CAF50',
```

```
    paddingVertical: 12,
    borderRadius: 8,
    marginTop: 20,
  },
  watchlistButtonActive: {
    backgroundColor: '#FF6347', // Red for remove
  },
  watchlistButtonText: {
    color: '#fff',
    fontSize: 18,
    fontWeight: 'bold',
    marginLeft: 10,
  },
});

export default ItemDetailsScreen;
```

### 6.5. Displaying Watchlist on WatchListScreen

This screen fetches and displays the user's watchlist if authenticated. If not authenticated, it should prompt the user to sign in or register to access their watchlist.

Modify `src/screens/WatchListScreen.js` to conditionally render its content:

```
// src/screens/WatchListScreen.js
import React, { useEffect, useState, useCallback } from 'react';
import { View, Text, FlatList, ActivityIndicator, StyleSheet, Image, TouchableOpacity, Alert } from 'r
eact-native';
import { useAuth } from '../context/AuthContext';
import { useWatchlist } from '../context/WatchlistContext';
import { useNavigation } from '@react-navigation/native';
import { Ionicons } from '@expo/vector-icons';

const WatchListScreen = () => {
  const { isAuthenticated, loading: authLoading } = useAuth();
  const { watchlist, loading: watchlistLoading, error, removeItemFromWatchlist, refreshWatchlist } =
useWatchlist();
  const navigation = useNavigation();
  const [isRemoving, setIsRemoving] = useState(false);

  useEffect(() => {
    // Optionally refresh watchlist when the screen focuses
    const unsubscribe = navigation.addListener('focus', () => {
      if (isAuthenticated) {
        refreshWatchlist();
```

```
    }
  });
  return unsubscribe;
}, [navigation, isAuthenticated, refreshWatchlist]);

const handleRemove = async (itemId, mediaType, itemTitle) => {
  Alert.alert(
    'Remove from Watchlist',
    `Are you sure you want to remove "${itemTitle}" from your watchlist?`,
    [
      { text: 'Cancel', style: 'cancel' },
      {
        text: 'Remove',
        onPress: async () => {
          setIsRemoving(true);
          try {
            await removeItemFromWatchlist(itemId, mediaType);
            Alert.alert('Removed', `${itemTitle} has been removed.`);
          } catch (err) {
            Alert.alert('Error', err.message || 'Failed to remove from watchlist.');
          } finally {
            setIsRemoving(false);
          }
        },
        style: 'destructive',
      },
    ]
  );
};

const renderWatchlistItem = ({ item }) => (
  <TouchableOpacity
    style={styles.watchlistItem}
    onPress={() => navigation.navigate('ItemDetails', { itemId: item.itemId, mediaType: item.mediaTy
pe })}
  >
    {item.itemPosterPath ? (
      <Image
        source={{ uri: `https://image.tmdb.org/t/p/w200${item.itemPosterPath}` }}
        style={styles.itemPoster}
      />
    ) : (
      <View style={styles.itemPlaceholder}>
        <Text style={styles.itemPlaceholderText}>No Image</Text>
      </View>
```

```
      )}
      <View style={styles.itemDetails}>
        <Text style={styles.itemTitle}>{item.itemTitle}</Text>
        {item.itemReleaseDate && <Text style={styles.itemInfo}>Release: {item.itemReleaseDate}</Te
xt>}
        {item.mediaType && <Text style={styles.itemInfo}>Type: {item.mediaType}</Text>}
        {item.seasonNumber && <Text style={styles.itemInfo}>Season: {item.seasonNumber}</Text
>}
        {item.episodeNumber && <Text style={styles.itemInfo}>Episode: {item.episodeNumber}</Tex
t>}
      </View>
      <TouchableOpacity
        style={styles.removeButton}
        onPress={() => handleRemove(item.itemId, item.mediaType, item.itemTitle)}
        disabled={isRemoving}
      >
        {isRemoving ? (
          <ActivityIndicator color="#fff" size="small" />
        ) : (
          <Ionicons name="close-circle" size={24} color="red" />
        )}
      </TouchableOpacity>
    </TouchableOpacity>
  );

  if (authLoading || watchlistLoading) {
    return (
      <View style={styles.loadingContainer}>
        <ActivityIndicator size="large" color="#0000ff" />
        <Text style={styles.loadingText}>Loading Watchlist...</Text>
      </View>
    );
  }

  if (!isAuthenticated) {
    return (
      <View style={styles.container}>
        <Text style={styles.messageText}>Please log in to view your watchlist.</Text>
        <TouchableOpacity style={styles.loginButton} onPress={() => navigation.navigate('SignIn')}>
          <Text style={styles.loginButtonText}>Go to Login</Text>
        </TouchableOpacity>
      </View>
    );
  }
```

```
  if (error) {
    return (
      <View style={styles.container}>
        <Text style={styles.messageText}>Error fetching watchlist: {error.message}</Text>
        <TouchableOpacity style={styles.refreshButton} onPress={refreshWatchlist}>
          <Text style={styles.refreshButtonText}>Try Again</Text>
        </TouchableOpacity>
      </View>
    );
  }

  if (watchlist.length === 0) {
    return (
      <View style={styles.container}>
        <Text style={styles.messageText}>Your watchlist is empty.</Text>
        <Text style={styles.subMessageText}>Add movies, TV shows, seasons, or episodes from thei
r details pages!</Text>
      </View>
    );
  }

  return (
    <View style={styles.container}>
      <Text style={styles.screenTitle}>My Watchlist</Text>
      <FlatList
        data={watchlist}
        keyExtractor={(item) => item.$id}
        renderItem={renderWatchlistItem}
        contentContainerStyle={styles.listContentContainer}
      />
    </View>
  );
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#000',
    alignItems: 'center',
    justifyContent: 'center',
    padding: 20,
  },
  loadingContainer: {
    flex: 1,
    justifyContent: 'center',
```

```
    alignItems: 'center',
    backgroundColor: '#000',
  },
  loadingText: {
    color: '#fff',
    marginTop: 10,
    fontSize: 16,
  },
  messageText: {
    color: '#fff',
    fontSize: 18,
    textAlign: 'center',
    marginBottom: 10,
  },
  subMessageText: {
    color: '#ccc',
    fontSize: 14,
    textAlign: 'center',
  },
  loginButton: {
    backgroundColor: '#007bff',
    paddingVertical: 10,
    paddingHorizontal: 20,
    borderRadius: 5,
    marginTop: 20,
  },
  loginButtonText: {
    color: '#fff',
    fontSize: 16,
    fontWeight: 'bold',
  },
  refreshButton: {
    backgroundColor: '#28a745',
    paddingVertical: 10,
    paddingHorizontal: 20,
    borderRadius: 5,
    marginTop: 20,
  },
  refreshButtonText: {
    color: '#fff',
    fontSize: 16,
    fontWeight: 'bold',
  },
  screenTitle: {
    fontSize: 28,
```

```
    fontWeight: 'bold',
    color: '#fff',
    marginBottom: 20,
    marginTop: 50, // Adjust for header
  },
  listContentContainer: {
    paddingHorizontal: 10,
    paddingBottom: 20,
  },
  watchlistItem: {
    flexDirection: 'row',
    backgroundColor: '#1a1a1a',
    borderRadius: 8,
    marginBottom: 15,
    overflow: 'hidden',
    alignItems: 'center',
    width: '100%',
  },
  itemPoster: {
    width: 80,
    height: 120,
    resizeMode: 'cover',
  },
  itemPlaceholder: {
    width: 80,
    height: 120,
    backgroundColor: '#333',
    justifyContent: 'center',
    alignItems: 'center',
  },
  itemPlaceholderText: {
    color: '#888',
    fontSize: 12,
    textAlign: 'center',
  },
  itemDetails: {
    flex: 1,
    padding: 10,
  },
  itemTitle: {
    color: '#fff',
    fontSize: 16,
    fontWeight: 'bold',
    marginBottom: 5,
  },
```

```
  itemInfo: {
    color: '#ccc',
    fontSize: 12,
  },
  removeButton: {
    padding: 10,
  },
});

export default WatchListScreen;
```

---

## 7. Final Thoughts and Best Practices

- **Error Handling:** Implement robust error handling with user-friendly messages.
- **Loading States:** Use loading indicators for better UX.
- **Security:**
  - **Environment Variables:** Use environment variables for sensitive data (e.g., Appwrite Project ID, Endpoint).
  - **Appwrite Permissions:** Verify Appwrite collection and document permissions; document-level permissions in `watchlistService.js` are crucial.
- **Offline Support:** Consider local caching (e.g., `AsyncStorage`) for offline watchlist access.
- **Real-time Updates:** Explore Appwrite's real-time capabilities for watchlist updates.
- **Scalability:** Use Appwrite functions/webhooks for server-side logic as the app grows.
- **UI/UX:** Focus on smooth UX and immediate feedback.

This guide provides a foundation for Appwrite auth, database (watchlist), and React Context API in your movie listing app. Adapt examples to your UI/structure.